

5.Agents底层逻辑之训练模式 - 图文解说

从一次参数调整，到一个可运行的模型。

本教程保留三种训练模式、手写数字案例、张量与并行、灾难性遗忘、泛化与拟合、架构与框架、模型文件和部署的完整讲解。

1. [01. 从参数更新说起：随机梯度下降](#)
2. [02. 全量训练之前，先有好的训练数据](#)
3. [03. 全批量梯度下降：全部算完，再求平均](#)
4. [04. 小批量训练：把数据分成一批一批](#)
5. [05. 张量：把一批数据组织起来并行计算](#)
6. [06. 训练的目标，是得到各方都能接受的结果](#)
7. [07. 为什么打乱数据：先学会 1，却忘了 0](#)
8. [08. 泛化、过拟合和欠拟合](#)
9. [09. 网络架构是图纸，学习框架把图纸落成代码](#)
10. [10. 训练框架与边缘计算](#)
11. [11. PyTorch 与推理：使用训练好的参数跑前向传播](#)
12. [12. 模型文件、权重与部署](#)

[打开重点讲解幻灯片](#)

一张关键画面对应一段完整解说，按原视频时间推进。

01. 从参数更新说起：随机梯度下降



完整内容

我想说的是，设计神经元的时候，不是层数越多越好。层数越多、神经元过量，可能会产生过拟合：只认识训练的数据集，不认识外面的东西。这节课我们就把神经网络这一部分结束，后面要讲 Transformer。这一节主要补充一些杂七杂八的小概念和术语。

第一个是梯度下降的训练模式。上节课讲了梯度下降、反向传播。看演示，现在只有三个需要训练的层；输入层不算，它没有这里要调整的权重和偏置。一次反向传播以后，每层的权重、偏置这些参数该怎么下降、怎么增加，就明确了。接下来怎么办？是马上更新参数，还是采用别的方式？这里就有不同的模式。

第一种叫随机梯度下降。这是早期的模式，早期的一些示例代码里会见到；这里说的是每次只取一个样本的做法。就像演示里，只有一个样本输入，得到这次的结果后，马上更新参数，然后换下一个样本，再下一个样本，循环往复。

这种做法主要的问题，是容易导致梯度噪声过大。什么叫梯度噪声？回想一下梯度下降：小球往坑里滚，或者你站在山坡上，顺着坡度往下降。当时的前提是什么？是输入不变，你站在同一座“山”上。输入一变，山脉就变了；输入变了，最后点亮的结构也全变了。

你可以想象，在黑夜里伸手不见五指，找到一个坡度，确实往下降了一步。可接下来输入一换，对这个新输入来说，你可能又跑到山顶上去了。在这个坡度下你又下降，对另一个输入却可能升高。这里刷新一下，演示输入还是固定的；那就换一个数字，再换一个数字，重新跑一遍训练。为这个数字下降了，对另外一个数字来说，可能反而升高。

不同输入对应的整座“山”是不一样的。如果只拿单个样本来算，参数就容易东跑一下、西跑一下，来回震荡。这种现象，我们这里就叫梯度噪声。

画面说明

课程文档列出 Stochastic Gradient Descent (SGD)：每一步只取 1 个样本计算梯度并更新参数，梯度噪声大、收敛震荡。下方同时列出全批量与小批量两种方式，形成后续比较的框架。

02. 全量训练之前，先有好的训练数据



完整内容

取单个样本计算，就会有刚才所说的来回震荡问题。第二种做法是全批量梯度下降。要理解它，先看训练数据集：训练要标注很多很多数据，用不同的数据去调整模型的各种参数。

以这个例子来说，我们有很多手写数字。这里的图片例子来自一位国外作者的神经网络博客或教材，我一时没想起他的名字，就把教材中的这些图片拿来用了。网上有这样的数据库，有几万张不同的手写数字图片，已经标注好了，可以用来训练。

训练中一个很麻烦的点，就是能不能拿到好的训练数据。这是非常、非常麻烦的一件事。拿到训练数据集以后，我们才把它拿去训练。每次训练包含一次前向传播和一次反向传播，接下来再看这些样本算出的结果如何用于更新参数。

画面说明

手写数字网格被红色矩形整体框选，展示不同书写方式的数字样本。讲解没有确认数据库或教材作者姓名，正文也不补猜出处。

03. 全批量梯度下降：全部算完，再求平均



场景 3

完整内容

一次前向传播、一次反向传播，可以算出一个样本对应的调整结果。但只用单一样本更新，梯度噪声会很大。一种解决办法就是全量训练，把所有样本都拿来计算。

可以这样理解：先拿第一个数据，跑一次前向传播，得到预测值，再跑反向传播，拿到第一个样本对应的参数调整结果。此时不更新参数，不更新。接下来拿第二个，再拿第三个。等拿到所有样本的结果以后，全量求平均。

对于同一个权重，有的样本希望它升高，有的希望它降低，就把这些调整求平均。好比东拉西扯：你往这个方向拉一点，他往那个方向拉一点。这里是高维空间，不是只有三维，而可能有几百、上千、上万维。在这个空间里东拉西扯，最后得到一个合适的点。

当然，训练不是一蹴而就的。全量跑一次不够，就反复跑：跑一次全量训练，求平均，更新参数；更新以后，再跑一次全量训练，再求平均，再更新参数。又跑、又求平均、又更新，就这样重复。

那跑到什么时候？跑到怎么弄损失值都很难再降下去，就要看结果能不能接受。比如图片识别达到 98% 的准确率，能接受，那就训练完毕，不再训练。如果只有 60%，损失又降不下去，就要考虑整个网络架构的设计问题。什么叫网络架构？就是整个神经元体系的设计。简单举例，隐藏层太少，神经元太少，或者太多，都需要调试和考虑。这是模型训练、算法工程师要处理的事情，不是这里应用开发要深入解决的工作。

画面说明

演示界面上叠放多组参数调整面板，分别列出权重与偏置的增减数值。它用于说明不同样本对相同参数提出的调整，要汇总之后再更新；这些面板是讲解示意，并非一个完整训练日志。

04. 小批量训练：把数据分成一批一批



完整内容

这就是全量训练、全批量梯度下降。在这里的讲解中，我们把它看作很理想的方式，但它烧硬件、烧成本。大模型参数这么多，训练集又庞大，比如两百多个 TB 的训练数据，成本能受得了吗？而且训练很慢、很慢，所以大模型训练几乎不会直接采用这种方式。

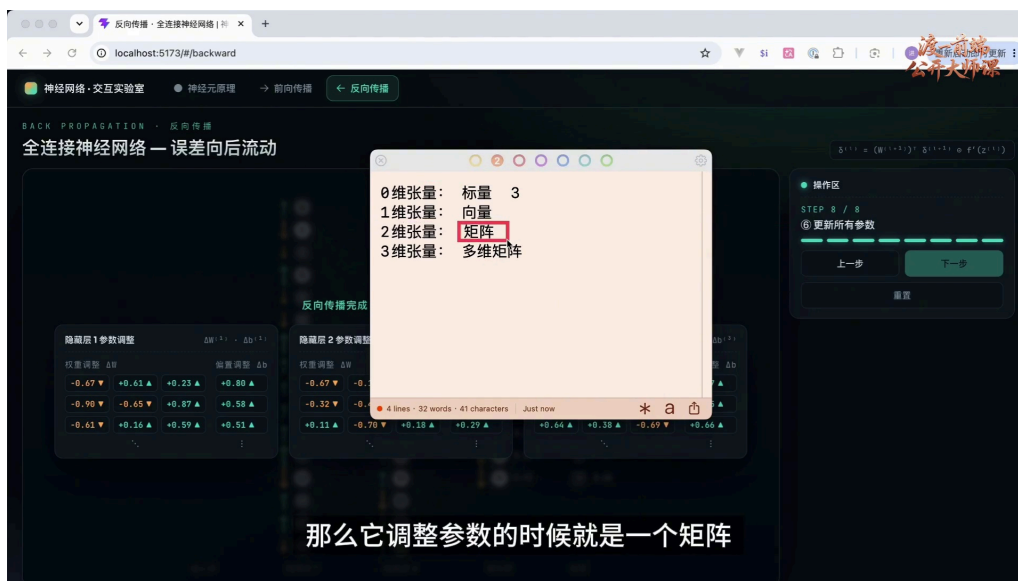
现在讲的主流方式是小批量梯度下降。怎么做？可以理解成切片：把这一块训练数据集拿过来，只跑其中这一批。为了演示方便，我先说跑第一个，再跑第二个、第三个，直到这一批全部跑完，再求平均，直接更新参数。更新之后，跑下一批，继续批量计算和更新。

这里要说明一下：为了让你们好理解，我是一个一个写、一个一个讲的，先跑第一个，再跑第二个。实际执行时，它们是一起跑的。接下来要讲的张量，就是理解这种批量并行的方法。

画面说明

数字网格的第一行被红框圈出，用于示意从完整数据集中抽取一个批次。课件写有 Mini-Batch Gradient Descent，举出的 batch_size 包括 32、128、1024，并说明它在稳定性、速度和显存之间取得平衡。

05. 张量：把一批数据组织起来并行计算



场景 5

完整内容

实际是怎么一起跑的？靠张量。它会把一个批次的数据组成张量。张量可以先想成几维数组：零维张量就是标量，就是一个数字，例如 3；一维张量是一个数组，也就是向量；二维张量是二维数组，也就是矩阵；三维张量，可以先理解为多维矩阵。

训练集有很多样本，每个样本可以是一个向量输入，参数的调整也可以按矩阵来组织，再往上就是多维的组织方式。这一块涉及数学，先了解就行。

为什么要弄成张量？在这里的解释里，它的运算规则与向量运算没有本质区别，GPU 仍然可以并行处理，一起完成。不是一个样本、一个样本串着跑，实际是一起跑。这个地方先了解这种批量计算的意思就可以了。

画面说明

便签完整列出“0 维张量：标量 3”“1 维张量：向量”“2 维张量：矩阵”“3 维张量：多维矩阵”，其中“矩阵”被红框圈出。背景是反向传播完成后的权重、偏置调整界面。

06. 训练的目标，是得到各方都能接受的结果



完整内容

一次跑一批，跑完更新参数，再跑下一批，再更新参数，然后继续往后。全部跑完，又从头来一轮，继续跑、继续更新。具体做法有很多，但这里先抓住一点：分批次更新权重和偏置，通过求平均，把不同样本的调整放在一起考虑。

这种“东拉西扯”意味着，训练模型并不是要求得到一个完美结果。完美结果很难达到，想让模型真正像人一样，也很难。尽管我们用计算机模拟人类的神经元，但人类神经元比计算机里的数学公式复杂得多、得多、得多。甚至到现在，人类也远远没有完全弄清楚神经元到底怎样工作，背后还有哪些东西。

所以人才是最复杂的，你才是最复杂的。计算机要用很多很多电，才能训练出一个模型；用这里的玩笑说法，你“两个馒头干两天”都可以。这是在强调人的复杂性，而不是要求计算机训练出绝对完美的东西。

训练的时候，在求平均、东拉西扯的过程中，你往这个方向拉一点，他往那个方向拉一点，拉到大家都能接受。照顾一下你，也照顾一下这位，这么多人都照顾一下。得到一个能接受的结果，就可以认为训练完成了。

讲到这种训练模式，还可以观察到一个特点：每一批训练数据里的数字，并不是整齐地按同一类别排在一起的。

画面说明

画面保留包含多种手写数字的网格，字幕为“大家都照顾一下”。图中各类数字混合排列，与不同样本的调整共同参与训练的比喻对应。

07. 为什么打乱数据：先学会 1，却忘了 0



完整内容

每批训练数据都随机打乱了，什么数字都有。为什么不先把 0 训练好？第一批全是各种各样的 0，第二批全是各种各样的 1，这样不行吗？这样可能产生另一个问题：灾难性遗忘。课件里原来没有专门写这个，我补充说一下。以后如果做微调，虽然未必像完整训练这么复杂，也可能遇到它。

假设先训练各种各样的 0。模型对 0 识别得非常准确、非常自信，但它没见过其他数字，权重和参数都向识别 0 的方向靠。这是一个极端结果：它只认识 0，反正把 0 识别好了。

接下来训练 1，各式各样的 1 对它来说就一塌糊涂。那肯定不行，还得重新训练，重新调整权重参数。调整了一大堆，终于把 1 识别好了。可是 1 识别好了，0 忘了，0 又出问题了，再去识别 0，又不行了。懂这个意思吧？这就叫灾难性遗忘。

因此，训练时往往打乱顺序。还是刚才那个比喻：你往这边拉一点，他往那边拉一点，拉到一个大家都能接受的结果。模型训练在这里就是这样理解的。

画面说明

便签写有“灾难性遗忘”，数字网格中的若干个 1 被红框圈出。画面用先关注 0、再关注 1 的例子说明连续训练不同类别时旧能力可能受到影响。

08. 泛化、过拟合和欠拟合



完整内容

再顺便说两个概念，平时讨论模型可能会提到：过拟合和泛化。先说泛化。怎么判断一个模型是不是智能？比如训练数据集里没有这张图片，但模型仍然认识它，仍然能识别出它的数字，我们就说它的泛化能力强。它能识别一些自己没见过的东西。

反过来，如果只认识训练集里的东西，到了训练集以外就一塌糊涂，这里就叫过拟合，也就是泛化能力弱。这种模型训练的结果就不理想。

还有欠拟合。欠拟合不是“只会训练集”的问题，而是连训练数据集都还没搞定，训练集里的东西都没认识清楚。

为什么讲这些？我想说，设计神经元时不是层数越多越好。在这里的例子中，层数、神经元过多，可能导致过拟合，只认识训练数据集，不认识外面的；过少，又可能导致欠拟合，能力不够，连训练集都吃不下。所以要把设计调整到一个合适的值。

画面说明

便签依次记录“灾难性遗忘”“过拟合”“泛化”“欠拟合”。背景仍是三种梯度下降训练模式的说明；随后演示界面用于指代隐藏层和神经元的数量。

09. 网络架构是图纸，学习框架把图纸落成代码



完整内容

所以，要调整到合适的值。这就是梯度下降训练模式这一部分。这节课主要是把这些零散概念给大家说一下。

接下来是网络架构。前面讲的是神经网络的一种通用做法，并不是唯一的标准做法。比如隐藏层设计多少层，每层多少神经元，都需要设计。层与层的连接一定是全连接吗？不一定，也可能只连接其中一部分。这些具体细节需要设计，不同设计就产生不同的网络架构。

这里介绍两种常见架构。一个是卷积神经网络，过去主要处理图片。课件把 CNN 写作图像领域的经典架构，用滑动的小窗口，也就是卷积核，提取局部纹理、边缘和色块，不用一次看完整张图。

另一个是 Transformer，后面会专门讲。它最初用于 NLP，也就是语言处理这一类任务，后来又用到图片、音频、视频等方面。本课把 Transformer 作为目前的主流方向来介绍，并把卷积网络称作较早期的做法。这里先了解架构这个概念。

有了架构，还要有学习框架。架构就像一张图纸：算法工程师告诉你整个神经网络怎么设计。拿到图纸以后，还得真正实施，变成代码，运行起来。学习框架也就是这里所说的训练框架，训练不就是学习吗？它负责把设计真正落实到代码层面。

画面说明

课件列出 CNN 和 Transformer。CNN 的说明是用卷积核提取局部纹理、边缘、色块；Transformer 的说明是“NLP 起家，现在通用全能架构”，并标注后续讲解。上述流行程度判断保留课程语境，不视为独立核实的行业统计。

10. 训练框架与边缘计算



完整内容

到代码落地的时候，工程师就开始干活了。图纸已经有了，戴上施工帽，开始敲敲打打，终于可以真正实施了。

这里说两套比较知名的训练框架。一个是 TensorFlow，过去很知名；按本课的介绍，它现在较少用于所讨论的大模型训练，但还会出现在一些边缘计算场景中。

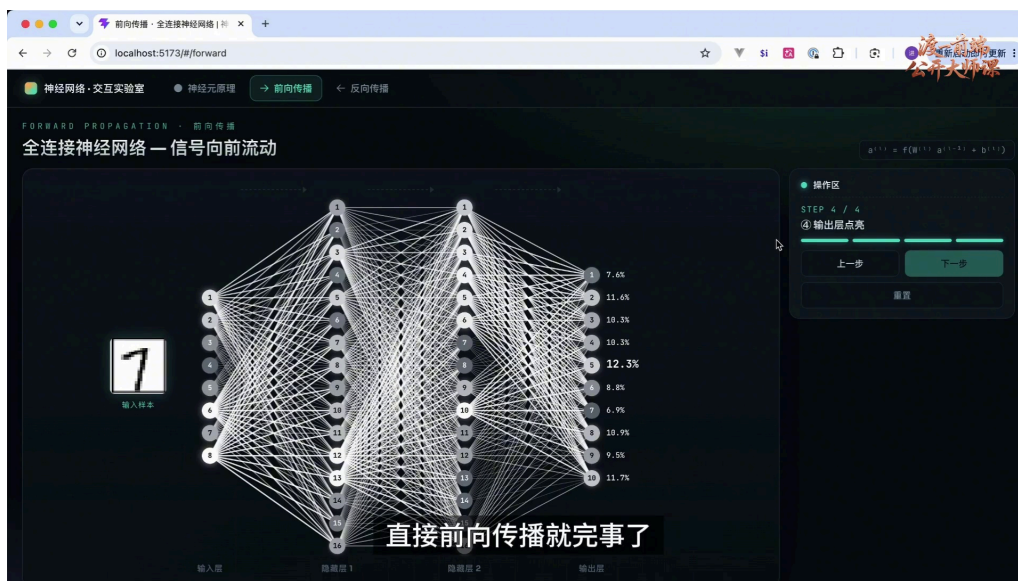
什么叫边缘计算？一般来说，大模型跑在云端。你不可能就在家摆一套 GPU 集群、搭建训练集群。那要正规训练机房、数据中心来做。

但是，有一些很小的训练任务或推理任务，计算量不大，很小、很迷你，在手机上就可能完成。这个时候，可以用 TensorFlow。这是这里举出的边缘计算例子。至于本课讨论的大模型训练学习，接下来主要介绍另一套框架。

画面说明

文档“深度学习框架”下并列列出 TensorFlow 与 PyTorch，分别写有边缘计算和主力训练框架的课程说明。页面下方开始出现“模型”标题。

11. PyTorch 与推理：使用训练好的参数跑前向传播



场景 11

完整内容

另一套就是 PyTorch。课件把它称为现在的主力训练框架，很多大模型通过它来训练。这就是学习框架，是真正落到代码、可以执行的东西。

这两个框架不只是训练框架，也可以做推理。什么是推理？可以先简单理解为：把前向传播走一遍。

训练时，为什么前向传播后还要反向传播？因为预测结果有问题，我们要调整参数。等模型的权重和偏置调整好了、训练完成以后，要怎么用这个模型？把这些参数填进去，直接做前向传播，得到结果，不需要再做后面的参数调整过程。这个过程就是推理。

训练框架既然能训练，就包含前向传播，也包含反向传播。训练好之后，我只使用它的前向传播过程就可以了。可以简单理解，前向传播就是这里所说的推理过程，也就是出结果的过程。

画面说明

神经网络交互演示显示输入数字 1、多个神经元层和层间连线，字幕为“直接前向传播就完事了”。这里借已有演示说明使用固定参数得到输出，而非展示一个新的训练步骤。

12. 模型文件、权重与部署



场景 12

完整内容

你输入，然后出结果。以前在训练阶段，还要接着反向传播和调整；训练好了以后，使用时就不需要这个调整过程。

接下来就可以解释“模型”到底是什么了。模型是训练的结果，参数调整好了，就产生了这个结果。它通常以多个文件的形式存储。

里面最核心的是权重文件，也可以叫核心参数文件。里面都是参数，包括权重和偏置。本课用“几万亿个权重、偏置”形容大模型的参数规模，并说权重文件占 99% 的体积，意思是大部分体积都被这些参数占用。训练的结果，就是产出这些权重和偏置。下次要使用，把它们读出来，放进网络里，跑一次前向传播，就可以用这个模型了。

除了权重，还有一些配置文件。光有权重参数不够，还要知道网络使用什么架构，有多少层，每层有多少神经元，才能复现网络结构。隐藏层维度、权重精度等额外信息，也需要配置文件描述。权重可能带小数，精度涉及怎样表示这些数值；这里先知道配置需要记录这类信息。

这些文件合在一起，就形成了这里所说的模型。后面我们会下载一些本地模型来玩。到时会发现，模型文件大，主要就是权重参数占体积，其他配置文件几乎不占什么体积。

下载后，可以直接用 PyTorch。它既是训练框架，也可以作为推理框架：把前向传播走一次，得到结果。后面本地运行模型，就会用到它。

不过，正经生产环境还要考虑性能问题。本课在这里强调，本地开发、测试、调试的做法，与生产部署的选择不完全一样。生产环境的性能、部署等问题，由部署工程师，也就是前面提到的 AI infra 工程师来处理，里面还有很多事情。

我们在本地玩模型，开发、测试、调试阶段，先这样使用就可以了。这节课就是补充这些专业术语，便于大家讨论；与这里要做的 AI 应用开发没有特别直接的关系。

画面说明

“模型”部分写明模型是训练结果，通常由多个文件组成：核心权重文件占主要体积；配置文件描述网络架构、隐藏层维度、权重精度等信息。画面同时保留 TensorFlow 与 PyTorch 两个框架名称。